清華大學
Tsinghua University

2026秋季

计算机系统概论

Introduction to Computer Systems

Zhang，Youhui（张悠慧）
zyh02@tsinghua.edu.cn











课程引言

参考书目：《深入理解计算机系统（第三版）》
(Computer Systems: A Programmer's Perspective)

课本内容对照要求（随着进展，可能略有变化）：

章节	要求的内容	说明
第二章	2.1；2.2；2.3（乘/除不要求，但2的整数次幂除外）；2.4 （2.4.5的运算不要求）	相关C语言概念要掌握，不专门讲
第三章	3.2；3.3；3.4；3.5；3.6；3.7；3.8；3.9；3.10	
第七章	7.1；7.2；7.3；7.4；7.5；7.6；7.7；7.8；7.9；7.10；7.12.1	
第八章	8.1；8.2；8.3；8.4；8.5；8.6	
第九章	9.1；9.2；；9.4；9.5；9.6（多级页表不要求）；9.7.2；9.8；9.9（9.9.12之前，包括实验）	
第十章	10.1；10.2；10.3；10.4；10.6；10.7；10.8；10.9；10.10；10.11	
第十二章	12.3；12.4；12.5（12.5.5不要求）；12.7	

联系方式：自强楼1-509；zyh02@tsinghua.edu.cn

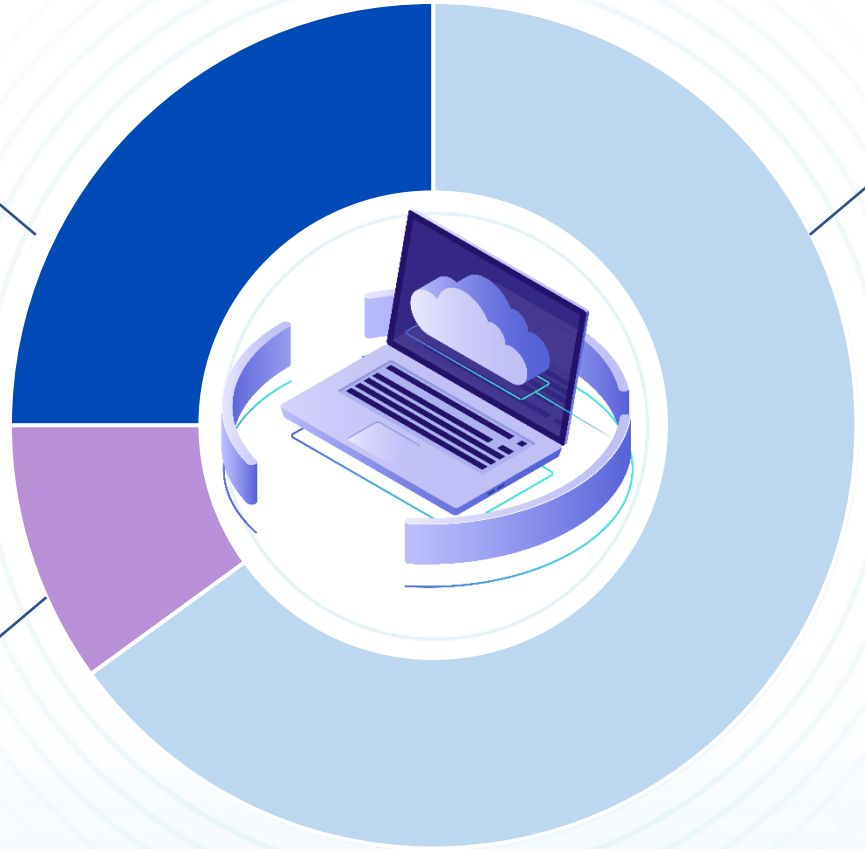
计分

实验

~30%

考试

~ 60%



作业

~ 10%

关于本堂课

01 关于计算机系统自身的课（相对于算法、应用等）

课程定位

- 系统类的引导课
  与其它系统课程的关系
- 大模型时代下的系统视角
- 课程知识点

02 课程的一些重要概念

- 计算机体系结构
- 程序；编译与链接

03 课程章节安排

- 课程章节安排
- 重要的设计思想

课程定位

计算机系统概论

本科专业主修课

大二秋季学期 / 课内学时48（3学分）

定位

系统课组入门课



课程学习目标

C / C ++ 语言程序设计

计算机编程者 使用者的视角

计算机系统概论

构建程序、进程以及初步的计算机系统概念，掌握系统硬件层面的程序表示与运行过程，并具备一定的系统编程能力，为学习后续的计算机系统课程打下基础

计算机组成原理

冯·诺依曼结构 / 计算机指令系统 / 汇编指令与编程

体系结构

线程与并发概念

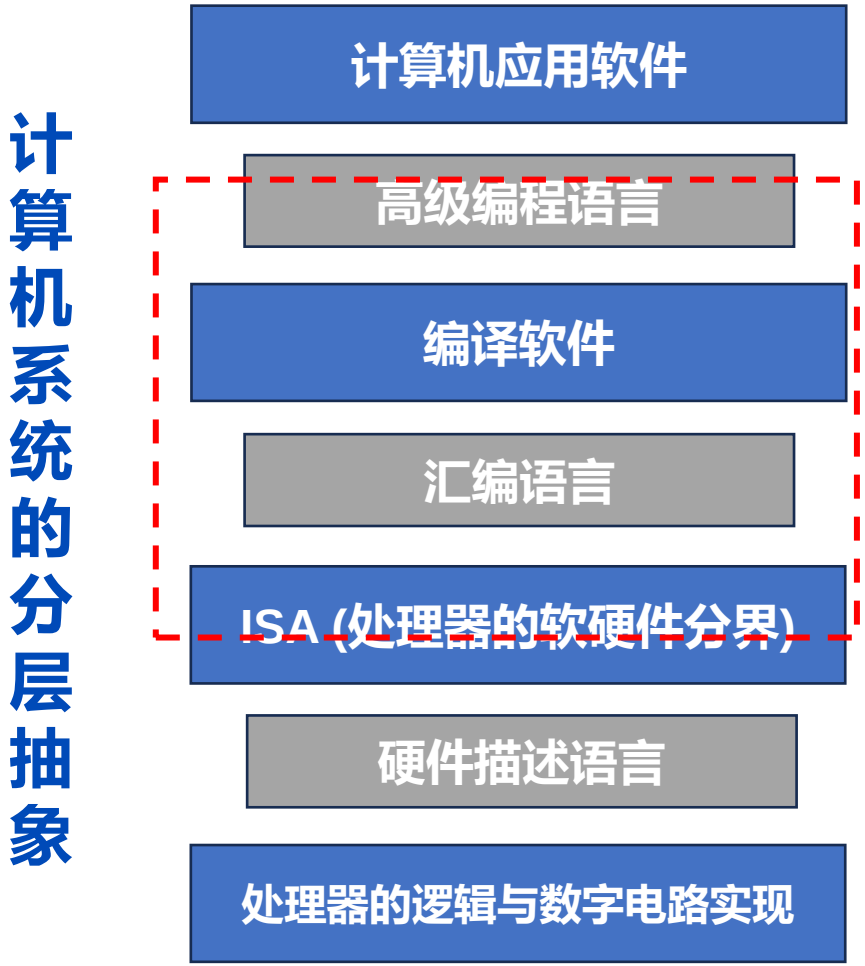
编译原理

数据与程序的机器表示 / 程序链接 / 编译优化（示例）

操作系统

异常处理 / 虚拟存储管理 / 系统调用

AI能写代码，为何还要学本课程?



我们在哪儿?

掌握了高级计算机编程语言（比如C/C++）

大模型做的如何？很好，很大程度可替代

要往哪儿去?

学习自上而下贯穿计算机软硬件层次的系统思维

大模型做的如何？目前擅长单层解释，但是代码设计的跨层权衡、问题情境判断需要人

怎么学?

跟随程序的生命周期，逐层深入，掌握系统思维

跨层C代码示例

```
…
struct A {
    char  c1;   // 1字节
    int   i;    // 4字节
    char  c2;   // 1字节
};
struct B {
    char  c1;   // 1字节
    char  c2;   // 1字节
    int   i;    // 4字节
};
struct C {
    char  c1;
    char  c2;
    int   i;
} __attribute__((packed));  // GCC: 紧凑排列/取消对齐
```

```
zhang@ubuntu:~/work$ gcc -O2 yinzi.c
zhang@ubuntu:~/work$ ./a.out
sizeof(struct A) = 12
sizeof(struct B) = 8
sizeof(struct C) = 6
```

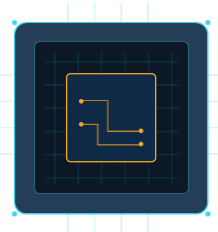
```
int main() {
    printf("sizeof(struct A) = %zu\n", sizeof(struct A));  // 典型输出: 12
    printf("sizeof(struct B) = %zu\n", sizeof(struct B));  // 典型输出: 8
    printf("sizeof(struct C) = %zu\n", sizeof(struct C));  // 输出: 6
    …
```

AI能告诉你 sizeof(A)=12, sizeof(B)=8，能解释数据对齐概念，但是——

- 当你面对一个遗留代码库，需要优化内存占用时，AI不会自动扫描所有结构体并重新排序字段——这需要自己理解为何排序重要、如何权衡（空间 vs. 访问局部性）

AI能解释现象，但优化决策需要你自己的判断

现实中更为复杂的实例——应用的跨处理器架构迁移



- 应用代码一般被认为是不依赖于处理器架构（如x86 / ARM等）的，但是“跨处理器架构迁移”的时候，并不是直接重新编译源代码就可以解决
- 一个大型央企进行业务系统迁移的时候，在**体系结构相关的“内存一致性模型”、“向量指令”、“原子操作与无锁编程”等方面遇到了大量语义方面的问题**——编译可以通过，但是执行错误——大模型也难以处理

看待计算机系统的角度

从编程者、使用者的角度入手

- 《计算机组成》/《计算机系统结构》等课程：从计算机构造者的角度入手

- 将高级语言程序/代码作为**理解整个计算机系统的起点**

- 从“计算机编程者、使用者”视角进行内容组织与调整，为具有一定C语言编程能力的本科生构建程序、进程以及初步的计算机系统概念

  掌握从C/C++语言到汇编语言，再到程序生成这一过程中所解决的基本问题，进程产生/运行/结束这一过程中的基本概念与处理流程

- 从而理解与掌握硬件层面的数据表示/程序表示与运行，并具备一定的系统编程能力

看待计算机系统的角度——大模型时代的新视角

从编程者、使用者的角度入手

- 《计算机组成》/《计算机系统结构》等课程：从计算机构造者的角度入手

- 将高级语言程序/代码作为**理解整个计算机系统的起点**
- 从“计算机编程者、使用者”视角进行内容组织与调整，为具有一定C语言编程能力的本科生构建程序、进程以及初步的计算机系统概念

  掌握从C/C++语言到汇编语言，再到程序生成这一过程中所解决的基本问题，进程产生/运行/结束这一过程中的基本概念与处理流程
- 从而理解与掌握硬件层面的数据表示/程序表示与运行，并具备一定的系统编程能力

大模型时代，知识的获取/理解/基本运用（布鲁姆认知层级的前三层）的门槛显著降低，而**分析/评价能力，以及具有跨计算机系统层次的系统性思维变得尤其重要**——面对vibe coding生成的高级语言代码，穿透其抽象层次，理解抽象层背后的下层约束或者能力，进行各种现象的根因分析，是目前大模型难替代的能力

看待计算机系统的角度——业界对计算机人才能力的需求变化

系统能力在 Vibe Coding 环境下更为重要*

场景	不具备系统能力	具备系统能力
面对 AI 辅助的系统优化	系统慢了就问 AI 哪里可以优化，拿到一堆建议碰运气	先建立基准量化瓶颈，逆向诊断根因，评估实现复杂度规划迭代路径
面对多轮交互后的混乱	方案越来越复杂，陷在 AI 生成的细节里出不来	随时回到抽象层判断方向是否正确，识别 AI 引入的额外复杂性，定期验证与初始目标的偏差
面对 AI 无法解决的问题	换个问法再问，AI 还是不知道，彻底卡住	重新分层确定 AI 能帮的范围，拆解为 AI 可处理的子问题，识别是实现、还是设计、还是分析不足的问题

*谢旻晖，卞昊穹，柴云鹏。AI编程智能体深度赋能的计算机系统教学实验重构，CCEC 2026

看待计算机系统的角度——穿透软硬件抽象层次的示例

课程视角为“软件角度”

一部分是“从上往下”看，从程序员/编译器实现程序功能、优化程序性能的实际需求出发，从高级语言（C语言）的表示出发，来讲解与分析涉及到的硬件方面的概念、原理等

一部分是“从下往上”看，从处理器的变化/演进角度来“观察”软件的变化

看待计算机系统的角度——穿透软硬件抽象层次的示例1

一部分是“从上往下”：从程序员/编译器实现程序功能、优化程序性能的实际需求出发，或者从高级语言（C语言）的表示出发，来讲解与分析涉及到的硬件方面的概念、原理等

课程视角为“软件角度”

硬件约束改变代码示例——存储位置对齐（” 80X86汇编语言与C语言-4”部分）

对齐的一般原则

- 已知某种基本数据类型的大小为 K 字节
- 那么，其存储地址必须是K的整数倍
- X86-64的对齐要求基本上就是这样

为何需要对齐

- **计算机访问内存一般是以内存块为单位的，块的大小是地址对齐的，如4、8、16字节对齐等**
  - 如果数据访问地址跨越“块”边界会引起额外的内存访问

编译器的工作

- **在结构的各个元素间插入额外空间来满足不同元素的对齐要求**

编译器掩盖了这一细节，在高级语言层次无法体现——如果设计处理器芯片，会在底层软件开发方面带来性能甚至正确性问题

看待计算机系统的角度——穿透软硬件抽象层次的示例2

课程视角为“软件角度”

条件移动

- 语义: if (Test) Dest ← Src
- 1995年以后的x86处理器都支持此类指令
- GCC编译器在确保“安全”的前提下会使用它们

Why?

- 条件跳转指令对于现代流水线处理器的执行效率有很大的负面影响
- 条件移动指令可以避免这一现象

C代码

```
val = Test
    ? Then_Expr
    : Else_Expr ;
```

Goto风格的代码表示

```
result = Then_Expr ;
eval = Else_Expr ;
nt = !Test;
if (nt) result = eval;
return result;
```

新的机器指令的引入，如何带来软件（汇编代码）层面的变化——编译器也掩盖了这一细节，但是对其的掌握利于了解处理器微架构，以进一步优化程序性能

写代码的需求影响硬件设计示例——条件移动指令（” 80X86汇编语言与C语言-2”部分）

一部分是“从下往上”：从处理器的变化/演进角度来“观察”软件的变化

美国一流大学相关必修课情况

	MIT	UC Berkeley	Stanford	CMU
课程名称	Computation Structures	Great Ideas in Computer Architecture (Machine Structures)	Computer Organization and Systems (COS)	Introduction to Computer System (ICS)
开设专业	EE、CS	CS	CS	CS
课程描述	门电路→功能部件→单周期和流水线CPU；C语言→汇编→指令→过程→进程；并行、性能评价	C语言→汇编→指令；应用级并行→数据级并行→线程级并行→指令级并行→寄存器传送级硬件描述	C语言→汇编→指令→微体系结构；编译→链接→装入执行；程序性能优化、存储器结构与管理、并发和多线程、网络编程	C语言→汇编→指令→微体系结构；编译→链接→装入→执行；程序性能优化、存储器结构与管理、并发和多线程、网络编程-
教程**	未指定（讲义）	C"K&R"+ COD"P&H"+ wsc"B&H"	PP"B&O"+ C"K&R"	PP"B&O"+ C"K&R"
模型机	自定义（RISC）	MIPS(RISC)	IA32/IA64	IA32/IA64
助教人数	12	7	15	未列出
先行课程或要求	了解编程（函教式）和数据结构基础，具备电子技术基础知识	了解和掌握编程（函数式）和数据结构基础，具备电子技术基础知识	了解和掌握编程和数据结构基础，具备电子技术基础知识	了解和掌握编程（函教式)和数据结构基础，具备电子技术基础知识
实验内容	共8个实验,涉及门电路特性、ALU、图灵机、汇编、处理器设计、鼠标中断方式I/O等	共12个实验，4个大作业，涉及到Debug、EC2、MapReduse 、MIPS汇编、数据级并行、线程级并行、cache、虚拟存储管理等，大作业和实验成绩占60%	共8 个实验和7个大作业,涉及教据的表示、堆区分配、过程调用和栈的构成及使用、溢出、编译工具和编译优化等，大作业和实验成绩占60%	共7个实验,涉及数据的表示、 cache 、缓冲区溢出、过程调用及栈的构成与使用、堆的分配、代理设置等，实验成绩占50%
实验手段	各类模拟器	编程、云计算平台、模拟器	编程	编程
相关后继课程或教学内容	“数学系统设计”和“计算机体系结构及系统”等,涉及CPU设计实验和体系结构方面的模拟实验	“数字系统设计”和“计算机体系结构及工程”等,涉及CPU设计实验和体系结构方面的模拟实验，前者是同时为CS和EE的学生开设的课程·	后继课程为“ Digital Systems II ”，教材为COD"P&H"",主要是流水线CPU和存储系统设计，实验为用DHL设计CPU(CS学生不需要)	CS学生可选ECE开设的课程Introduction to Computer Architecture”，教材为COD""P&H"，主要是流水线CPU和存储系统设计,实验为用Verilog语言设计CPU

课程知识点

基础部分（程序/数据在机器层面的表示与运行）

基础概念：计算机体系结构；程序；编译与链接作用

编译相关：数据的机器表示；汇编基本指令；程序的机器表示

链接相关：程序的链接过程；内存布局；栈溢出攻击

计算机系统基础概念

异常与异常控制流、虚存与虚存管理、用户层动态内存分配、线程与并发

计算机系统编程

系统调用：进程管理、信号处理、文件IO、线程编程

实验初步安排（3个）

炸弹实验 or 栈溢出攻击（程序的机器表示、debug）| **动态内存分配**（空间和时间效率的取舍）| **多核调度器***（并发+任务状态）

内容组织：一个进程（程序）从诞生到消亡

C程序在硬件层面的表示
（单进程“假象”：全部内存与整个处理器）

在硬件层面的运行
（在并发多任务下，“假象”如何形成？）

编译

- 数据的机器表示
- 汇编基本指令
- 程序的机器表示

链接

- 程序链接
- 内存布局
- 栈溢出攻击

启动

- 创建虚存空间
- _start运行

（涉及简单汇编编程）

运行

- 虚存与虚存管理
- 异常与异常控制流
- 动态内存分配

退出

- 进程管理
  - 子进程exit
  - 父进程回收

其它涉及内容还有：信号处理；文件IO；线程与并发

内容组织：一个进程（程序）从诞生到消亡

C程序在硬件层面的表示
（单进程假象：全部内存与整个处理器）

在硬件层面的运行
（在并发多任务下，“假象”如何形成？）

编译	链接	启动	运行	退出
数据的机器表示	程序链接	创建虚存空间	虚存与虚存管理	进程管理
汇编基本指令	内存布局	_start运行（涉及简单汇编编程）	异常与异常控制流	子进程exit
程序的机器表示	栈溢出攻击		动态内存分配	父进程回收

其它涉及内容还有：信号处理；文件IO；线程与并发

跨层次思考——

谁提供了这个“假象”？→ 编译器 + OS + 处理器（三层协作）

用什么机制掩盖？ → 虚存（地址空间隔离）、异常（上下文切换）

掩盖的代价是什么？ → TLB miss、page fault、上下文切换开销等

何时这个“假象”会“破裂”？ → 性能异常等，比如memory thrashing（内存抖动）

课程的一些重要概念



计算机体系结构

基本概念（广义；狭义）



程序；

编译与链接

计算机体系结构 | 狭义概念；Instruction Set Architecture



IBM360

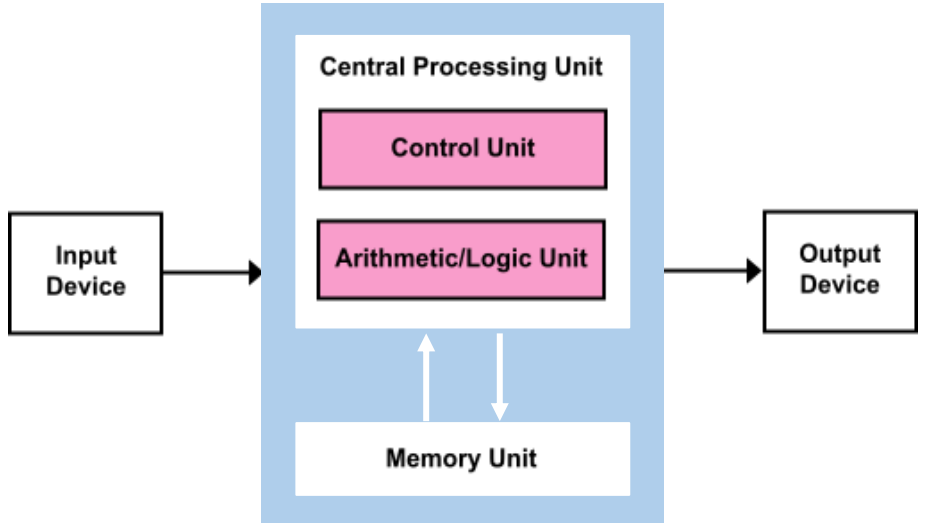
当G. M. Amdahl于1964年推出世界上第一个系列计算机IBM360时，把**Computer Architecture** 引入（**商业兼容性与软件生态**问题）

系统程序员所看到的计算机系统的属性，包括**概念性结构和功能特性**

- **程序员：**系统程序员（汇编语言、机器语言、编译程序、操作系统等）
- **看到的：**编写出能在机器上正确、高效运行的程序所必须了解的

***本课程涉及的计算机系统主要包含中央处理器与内存**

概念性结构——“冯•诺依曼架构”





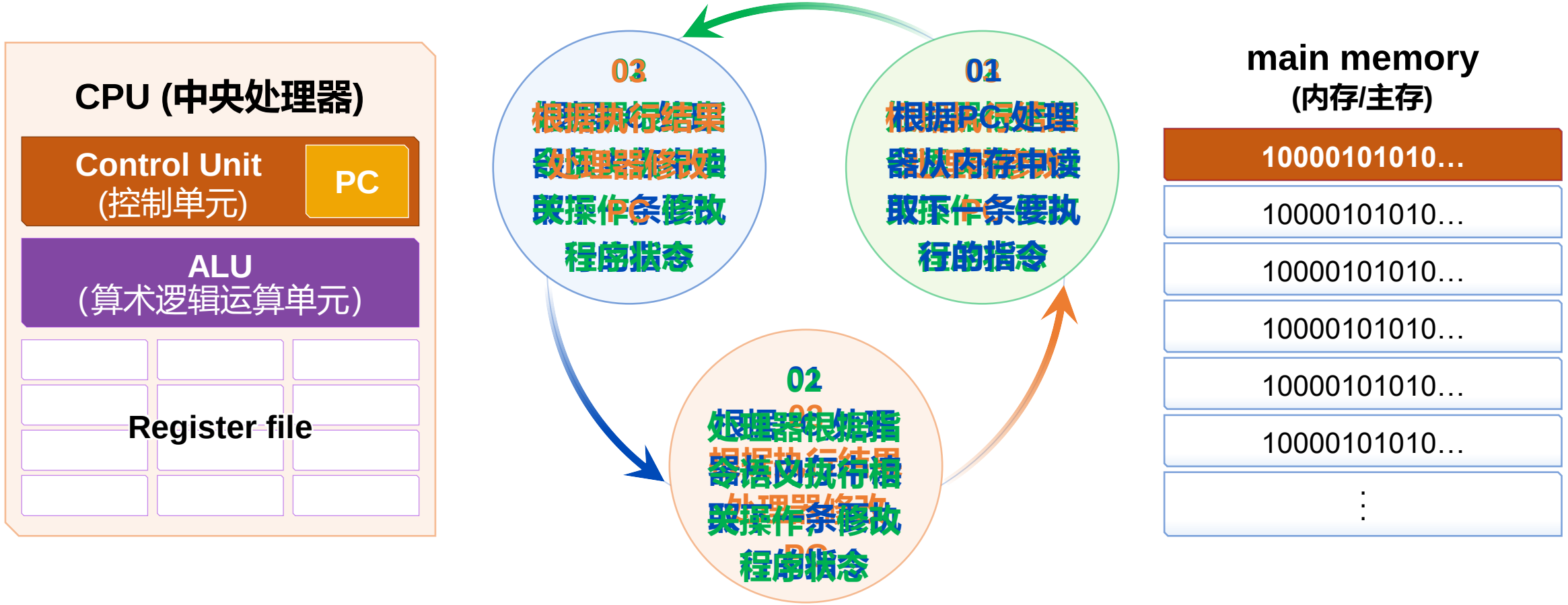
A design architecture for an electronic digital computer with parts consisting of a **processing unit containing an arithmetic logic unit and processor registers, a control unit containing an instruction register and program counter**, **a memory** to store both data and instructions, external mass storage, and input and output mechanisms.

John Von Neumann，1903 - 1957 (From wiki)

Long live the von Neumann architecture!

“冯•诺依曼架构”的指令执行示意

冯诺依曼架构的程序执行特征：以指令为中心 / 存算分离



program counter（程序计数器，缩写为PC）：记录下一条执行指令的地址

程序状态包括处理器状态，如register file（寄存器堆）等，与内存中的程序数据

程序代码与数据存储于此：代码以机器指令的形式存放,机器指令的助记符即为汇编指令

“冯•诺依曼架构”的指令执行示意

冯诺依曼架构的程序执行特征：以指令为中心 / 存算分离



program counter（程序计数器，缩写为PC）：记录下一条执行指令的地址

程序状态包括处理器状态，如register file（寄存器堆）等，与内存中的程序数据

程序代码与数据存储于此：代码以机器指令的形式存放,机器指令的助记符即为汇编指令

“冯•诺依曼架构”的指令执行示意

冯诺依曼架构的程序执行特征：以指令为中心 / 存算分离



program counter（程序计数器，缩写为PC）：记录下一条执行指令的地址

程序状态包括处理器状态，如register file（寄存器堆)、PC等，与内存中的程序数据

程序代码与数据存储于此：代码以机器指令的形式存放,机器指令的助记符即为汇编指令

功能特性——指令系统及其执行模式

数据表示	硬件直接识别和处理的数据类型
寻址技术	编址方式、寻址方式和定位方式
寄存器定义	寄存器定义、数量和使用规则
指令系统	指令的操作类型、格式、操作数类型等
存储系统	要求速度高、容量大、价格便宜
异常处理系统	异常类型、级别和响应方式
输入输出系统	控制数据交换方式与交换过程
机器工作状态	定义、切换工作方式，如内核态、用户态等

IBM Compatibility Problem in early 1960s

By early 1960's, *IBM had 4 incompatible lines of computers!*

701	7094
650	7074
702	7080
1401	7010



Each system had its own
- Instruction set
- I/O system and Secondary Storage: magnetic tapes, drums and disks
- Assemblers, compilers, libraries,...
- Market niche: business, scientific, real time, ...

IBM System/360 – one ISA to rule them all

IBM 360: A Computer Family

	Model 30	. . .	*Model 70*
Storage	8K - 64 KB		256K - 512 KB
Datapath	8-bit		64-bit
Circuit Delay	30 nsec/level		5 nsec/level
Registers	Main Store		Transistor Registers

The IBM 360 is why bytes are 8-bits long today!

IBM 360 instruction set architecture (ISA) completely hid the underlying technological differences between various models.

Milestone: The first true ISA designed as portable hardware-software interface!

With minor modifications it still survives today!

From "50 years of Computer Architecture ", by David Patterson, May, 2017



广义概念

广义概念

计算机体系结构(狭义)
Computer Architecture

程序员所看的计算机
系统的属性

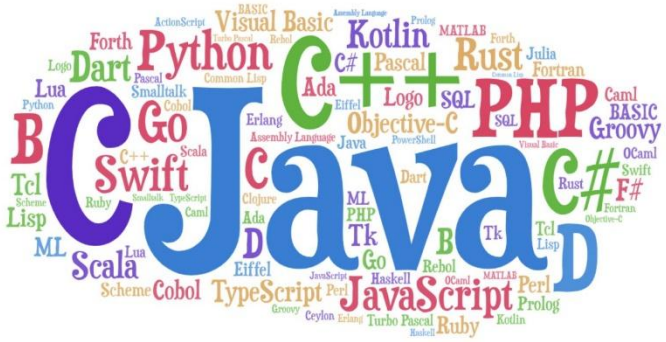
计算机组成
Computer Organization

计算机系统的逻辑实现

计算机实现
Computer Implementation

计算机系统的物理实现

课程的一些重要概念



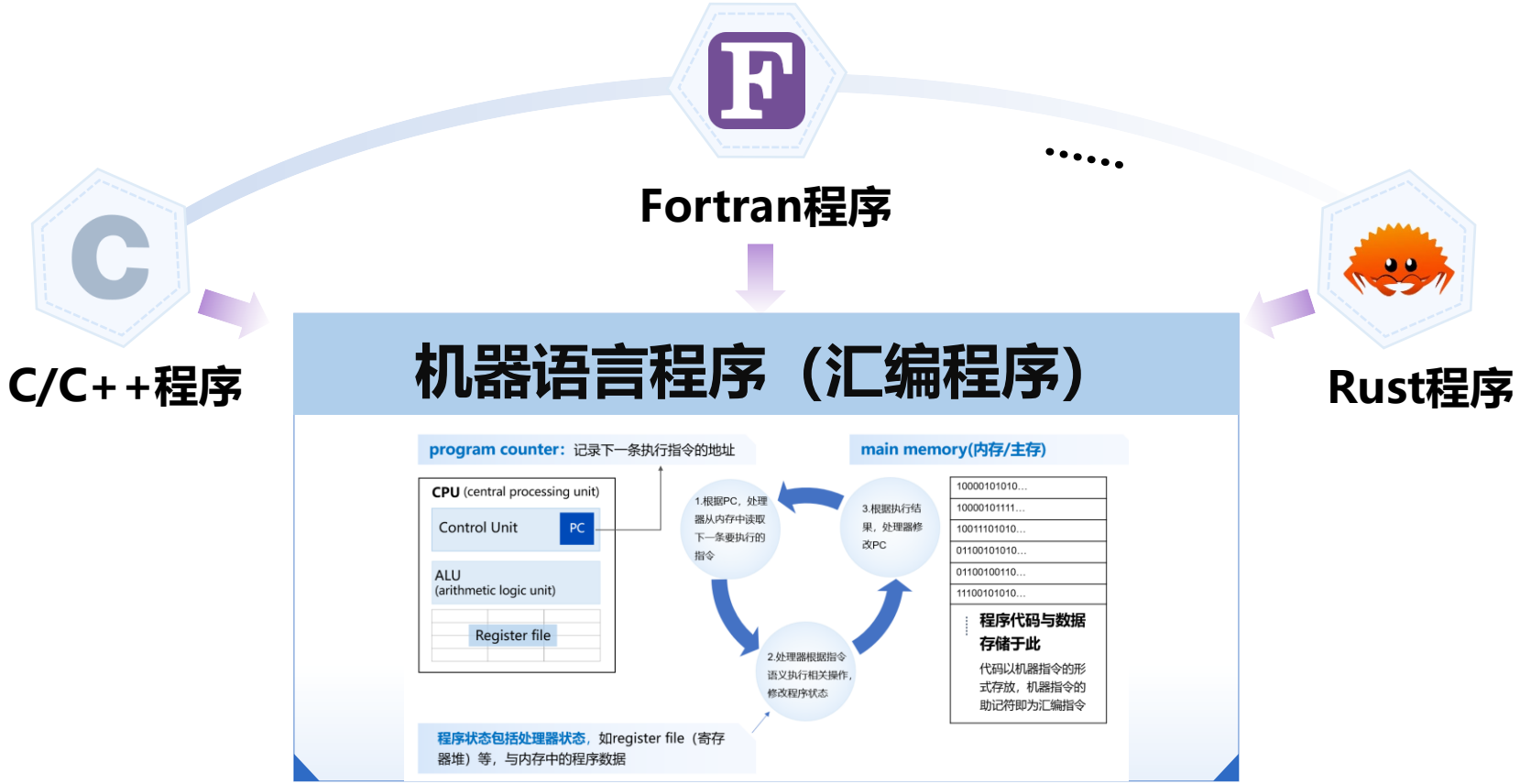
计算机程序



编译和链接

程序

计算机程序是一组告诉计算机要做什么的指令 (from wiki)



编译

```c
int array[4] = {1,2,3,4};
static int brray[4] = {1,2,3,4};

static int intra_sum (int x[4],int y)
{
   return x[y-1];
}
int main()
{
    int val = intra_sum(array, 3) + inter_sum(brray,3);
    return val;
}
```

C语言程序 (.c)

编译:把用一种编程语言编写的计算机程序翻译成另一种语言

(from wiki)

```
intra_sum:
        movslq  %esi, %rsi
        movl    -4(%rdi,%rsi,4), %eax
        ret
main:
        pushq   %rbx
        movl    $3, %esi
        movl    $array, %edi
        call    intra_sum
        movl    %eax, %ebx// val -> ebx
        movl    $3, %esi
        movl    $brray, %edi
        movl    $0, %eax
        call    inter_sum
        addl    %ebx, %eax// val -> eax
        popq    %rbx
        ret

brray:
        .long   1
        .long   2
        .long   3
        .long   4
array:
        .long   1
        .long   2
        .long   3
        .long   4
```

汇编指令

```
0000000000000000 <intra_sum>:
   0:   48 63 f6
   3:   8b 44 b7 fc
   7:   c3

0000000000000008 <main>:
   8:   53
   9:   be 03 00 00 00
   e:   bf 00 00 00 00
  13:   e8 00 00 00 00
  18:   89 c3
  1a:   be 03 00 00 00
  1f:   bf 00 00 00 00
  24:   b8 00 00 00 00
  29:   e8 00 00 00 00
  2e:   01 d8
  30:   5b
  31:   c3
```

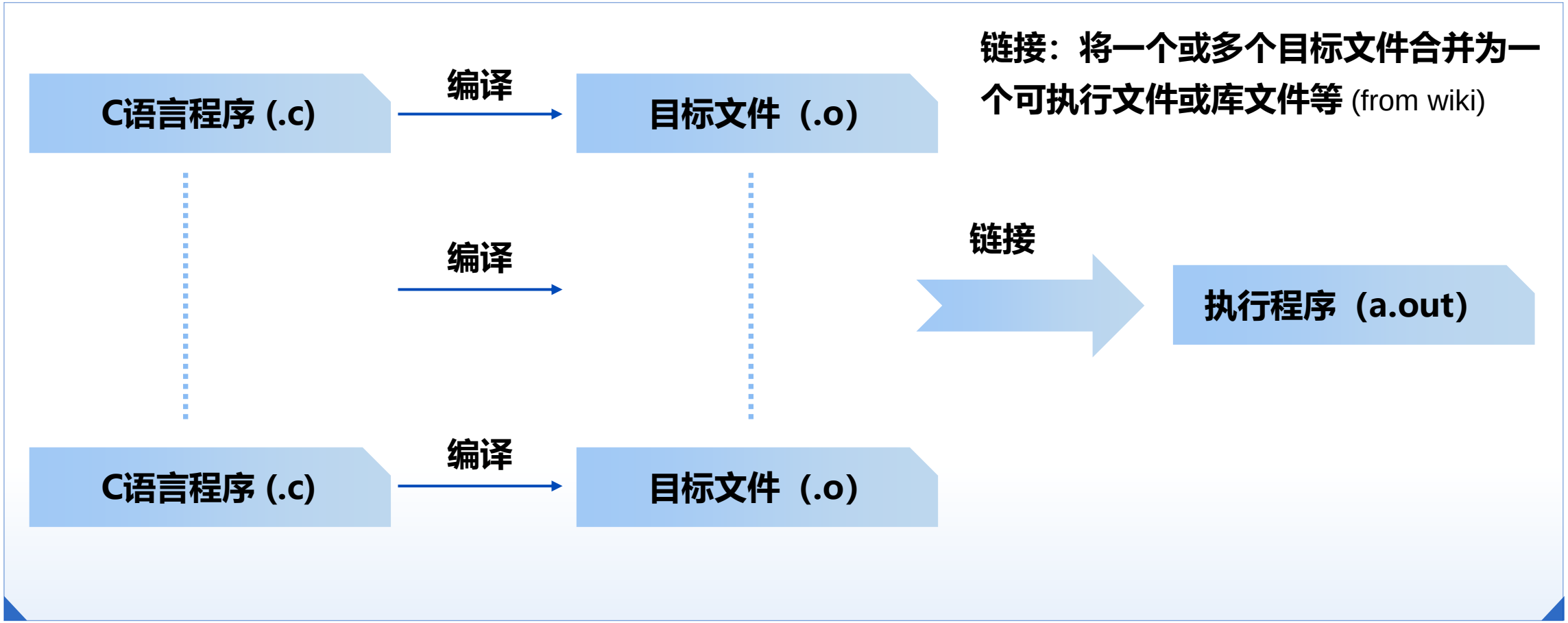
机器指令

数据部分没有显示

汇编程序（.s）

对象程序（.o）

链接



链接：将一个或多个目标文件合并为一个可执行文件或库文件等 (from wiki)

程序的编译、链接、运行

```c
int array[4] = {1,2,3,4};
static int brray[4] = {1,2,3,4};

static int intra_sum (int x[4],int y)
{
   return x[y-1];
}
int main()
{
    int val = intra_sum(array, 3) + inter_sum(brray,3);
    return val;
}
```

main.c

编译

```
intra_sum:
        movslq  %esi, %rsi
        movl    -4(%rdi,%rsi,4), %eax
        ret
main:
        pushq   %rbx
        movl    $3, %esi
        movl    $array, %edi
        call    intra_sum
        movl    %eax, %ebx// val -> ebx
        movl    $3, %esi
        movl    $brray, %edi
        movl    $0, %eax
        call    inter_sum
        addl    %ebx, %eax// val -> eax
        popq    %rbx
        ret
brray:
        .long   1
        .long   2
        .long   3
        .long   4
array:
        .long   1
        .long   2
        .long   3
        .long   4
```

汇编指令

main.o

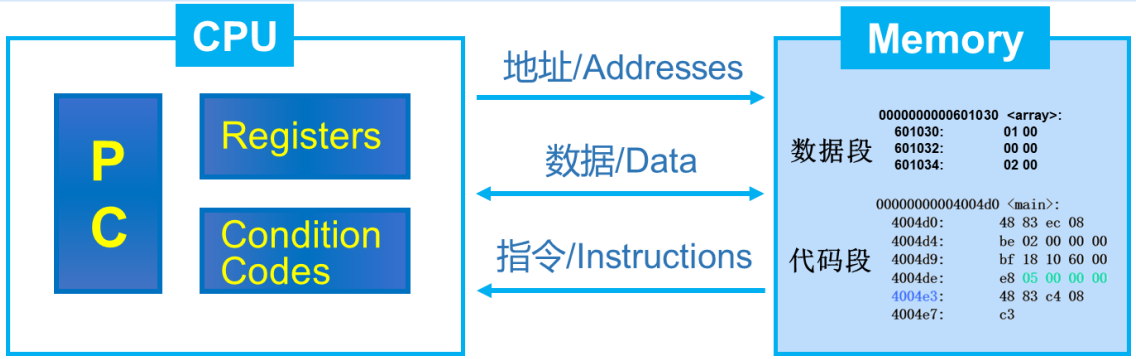
链接

```
00000000004004de <main>:
  4004de:       53                      push   %rbx
  4004df:       be 03 00 00 00          mov    $0x3,%esi
  4004e4:       bf 40 10 60 00          mov    $0x601040,%edi
  4004e9:       e8 e8 ff ff ff          callq  4004d6 <intra_sum>
  4004ee:       89 c3                   mov    %eax,%ebx
  4004f0:       be 03 00 00 00          mov    $0x3,%esi
  4004f5:       bf 30 10 60 00          mov    $0x601030,%edi
  4004fa:       b8 00 00 00 00          mov    $0x0,%eax
  4004ff:       e8 04 00 00 00          callq  400508 <inter_sum>
  400504:       01 d8                   add    %ebx,%eax
  400506:       5b                      pop    %rbx
  400507:       c3                      retq
```

内存地址

仅给出部分内容

运行

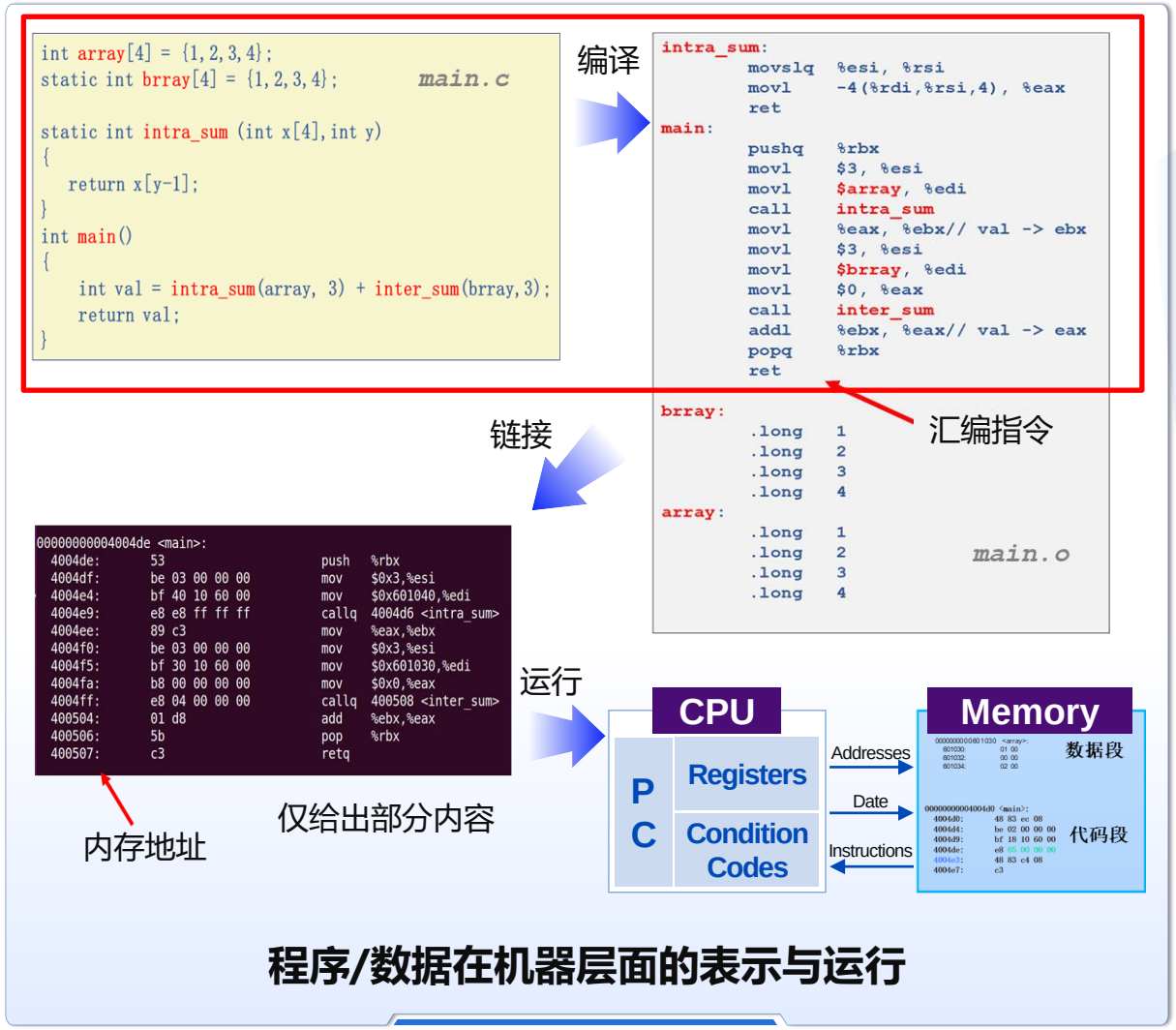


程序/数据在硬件层面的表示与运行



课程章节安排

课程章节安排



程序/数据在机器层面的表示与运行

C程序在硬件层面的表示

数据

- 整数 / 浮点数（第二讲）
- 数组 / 结构　（第五讲）

代码

- 基本概念/基本指令/寻址方式/程序控制流与相关指令（第三讲）
- 函数调用与相关指令（第四讲）

课程章节安排

C程序在硬件层面的表示

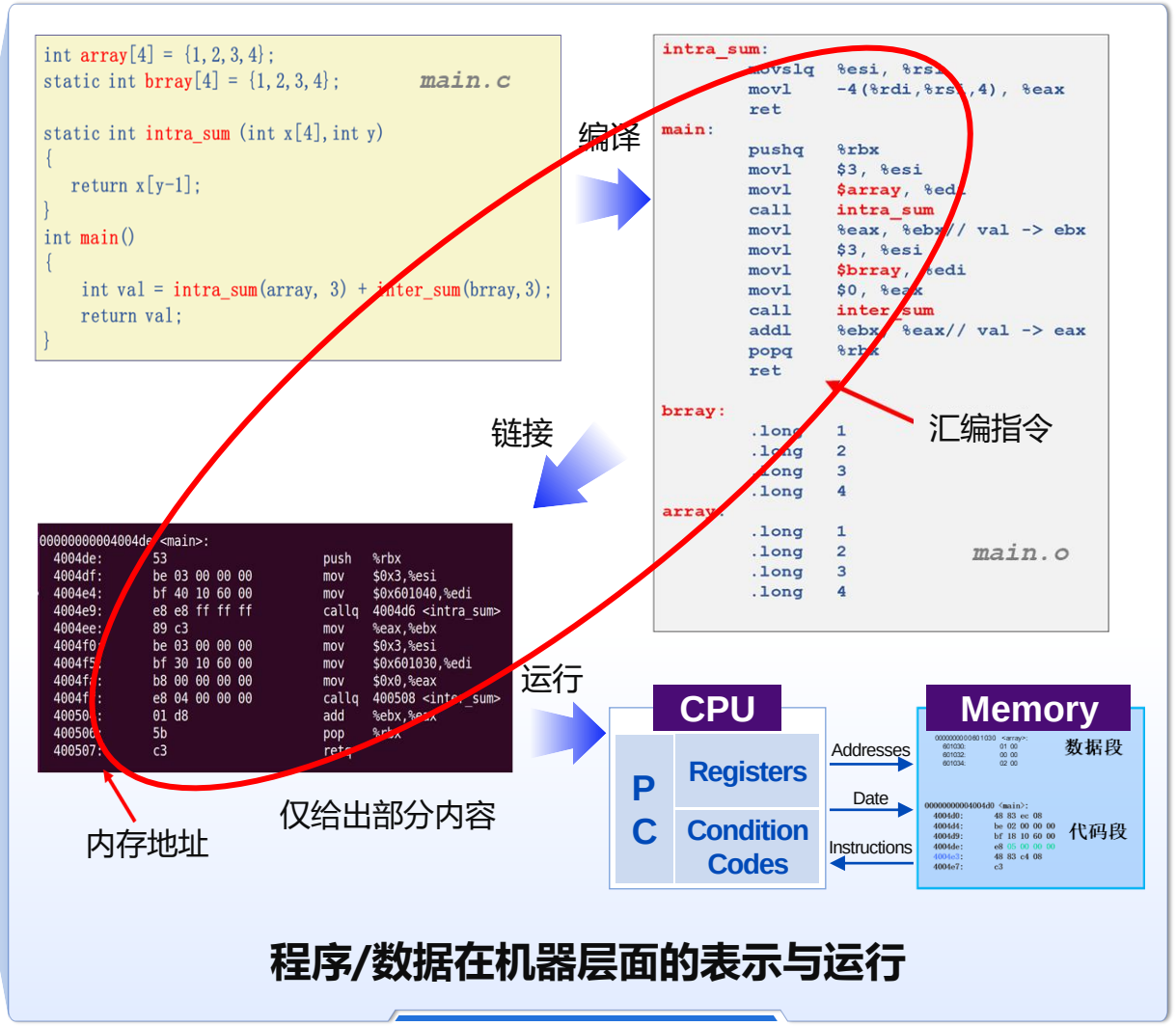
数据/函数的内存地址定位

- 链接（第六讲）

数据/代码的内存布局

- 栈、堆等各类数据段以及代码段的layout（第六讲）
- 缓冲区溢出等（第六讲）

讲解基本调试工具GDB等的使用

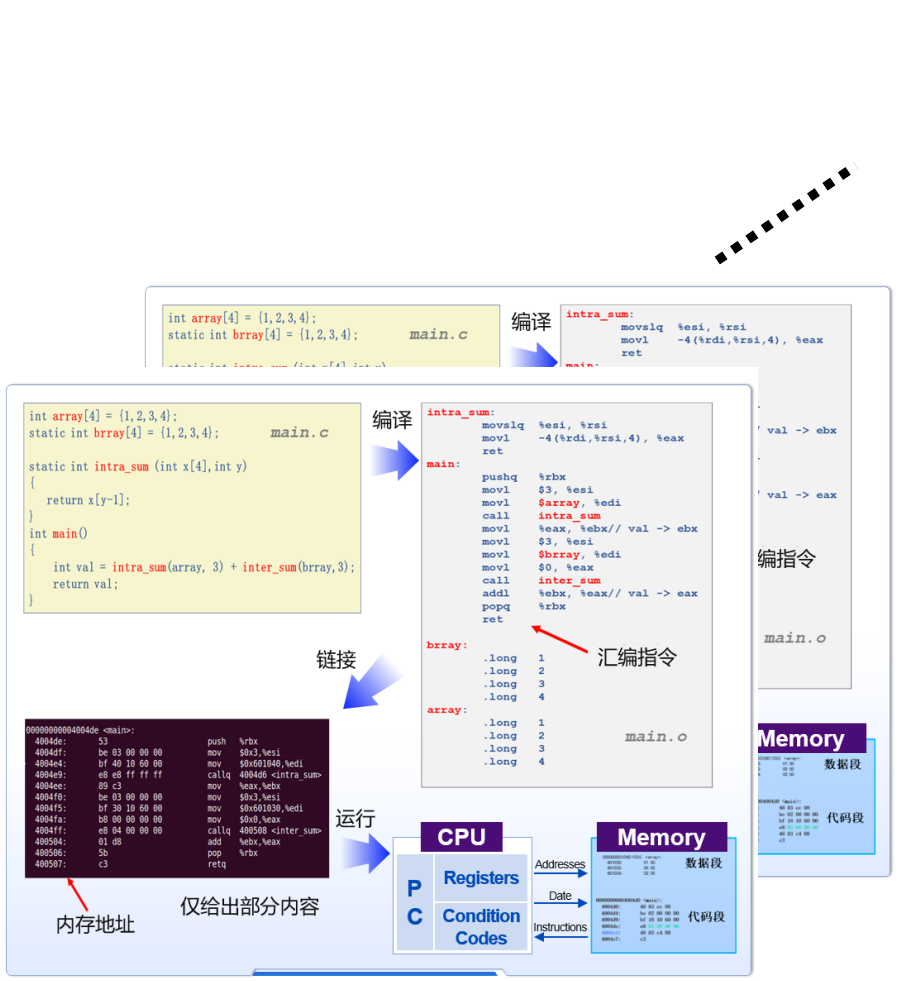


程序/数据在机器层面的表示与运行

课程章节安排

(看上去)

一个程序占据了整个处理器及一块内存空间，但是实际情况（多任务）要远为复杂!



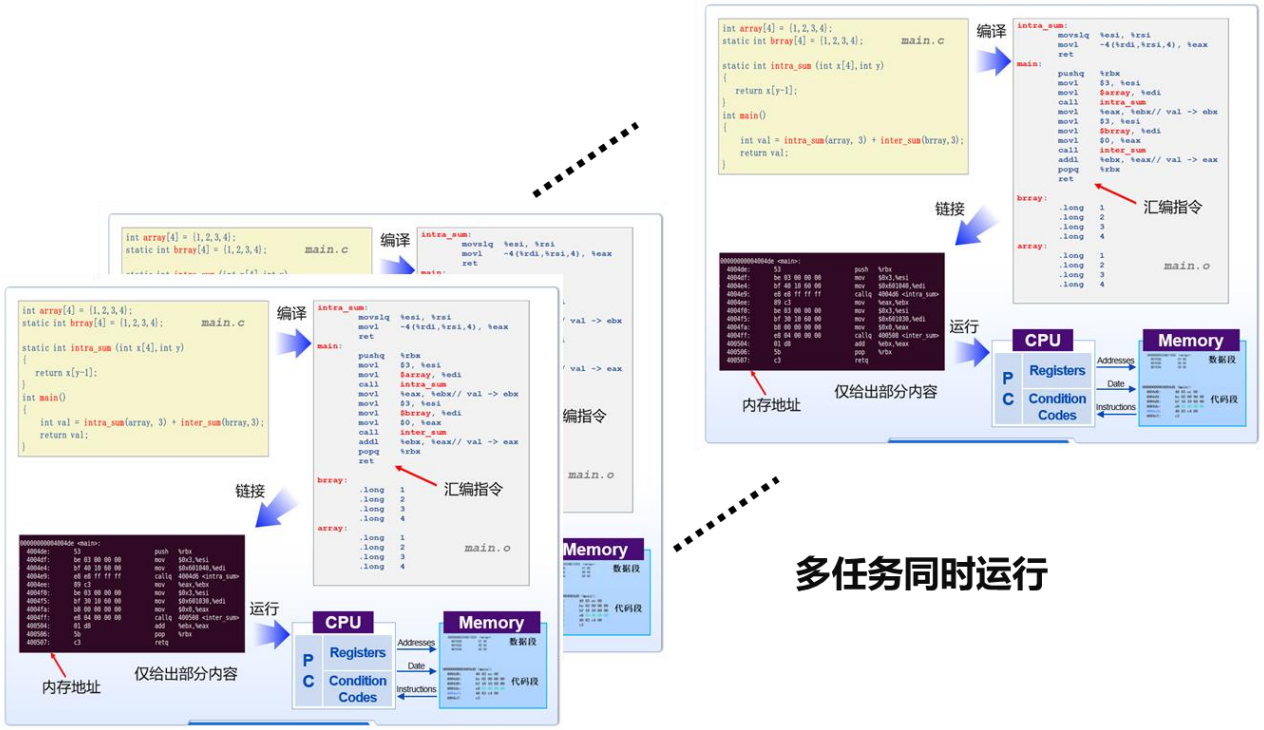
多任务同时运行

怎么做到的?

课程章节安排

(看上去)

一个程序占据了整个处理器及一块完整的内存空间，但是实际情况（多任务）要远为复杂！



怎么做到的?

关系到计算机系统的两个重要概念

- **虚存** - 在有限物理内存前提下设计出连续的、相互独立的虚拟内存
- **异常** - 各个任务切换的重要机制
  (当然异常还有很多其他作用)

课程章节安排

虚 存

在有限物理内存前提下设计出连续的、相互独立的虚拟内存

异常

任务切换的重要机制

（当然异常还有很多其他作用）

虚存与虚存管理（第七、八讲）

- 包括用户层动态内存分配

异常与异常控制流（第九讲）

- 进程上下文切换
- 进程层次结构
- 信号处理

其他内容（第十、十一讲）

系统调用：系统级IO、线程与并发、线程简单编程

重要的设计思想

计算机系统的分层抽象原则



计算机体系结构(狭义)
Computer Architecture

程序员所看的计算机系统的属性

计算机组成
Computer Organization

计算机系统的逻辑实现

计算机实现
Computer Implementation

计算机系统的物理实现

广义概念

硬件

计算机应用软件

高级编程语言

编译软件

汇编语言

ISA (处理器的软硬件分界)

硬件描述语言

处理器的逻辑与数字电路实现

重要的设计思想

计算机系统的设计思想通常很“朴素”*—— 第一性原理解决问题的典范

基于栈的函数调用

需求：函数调用要保存现场、层层嵌套

→ 构建一个“后进先出”的栈：调用就是 push，返回就是 pop，一条栈指针解决问题！

异常处理

需求：程序执行的正常流程被打断，保存现场以恢复

→ 硬件自动保存现场，通过异常处理向量表精准转移到软件处理入口

虚拟内存

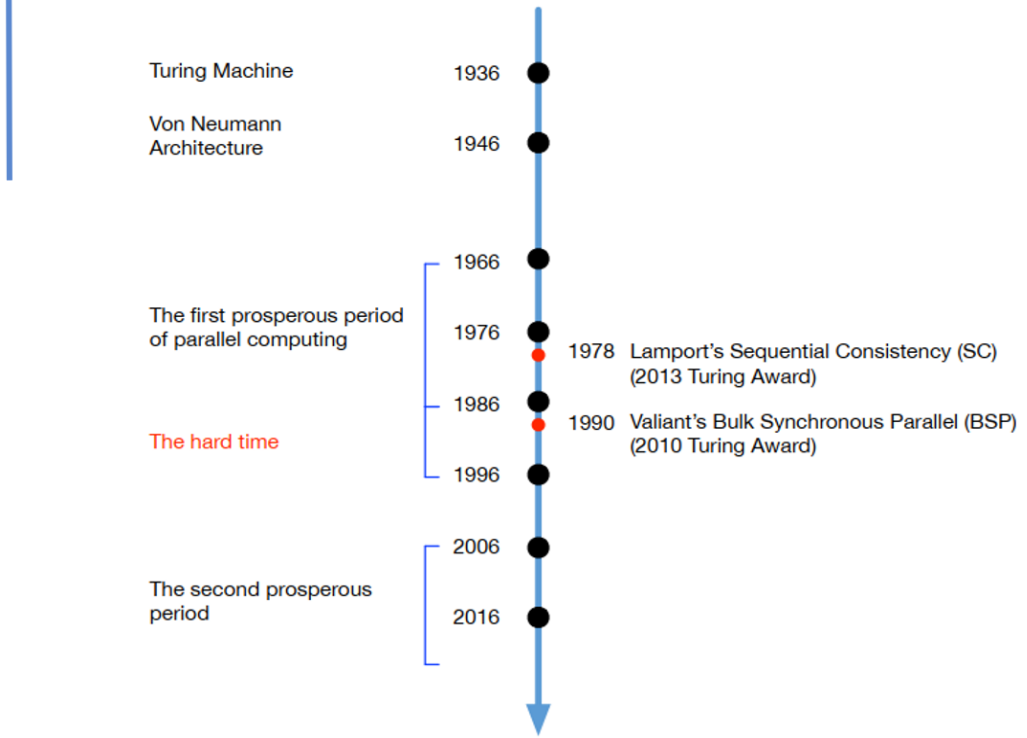
需求：内存不够用

→ 假装足够大，缺的时候再到硬盘现搬；页表+缺页异常就把“无限”与“有限”桥接起来

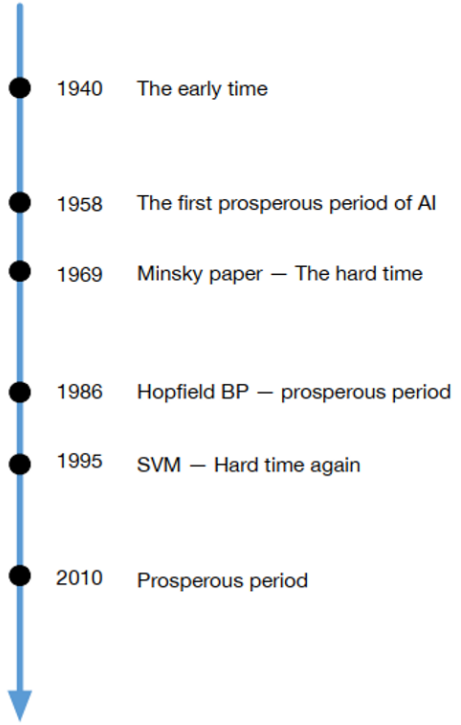
*思想朴素，不等于实现简单；先掌握核心思想，细节在后续章节以及课程展开

计算机系统发展对AI发展的重要性

计算机系统的重要性举例——AI发展历史与并行计算发展历史的比对



(a) Parallel Computing

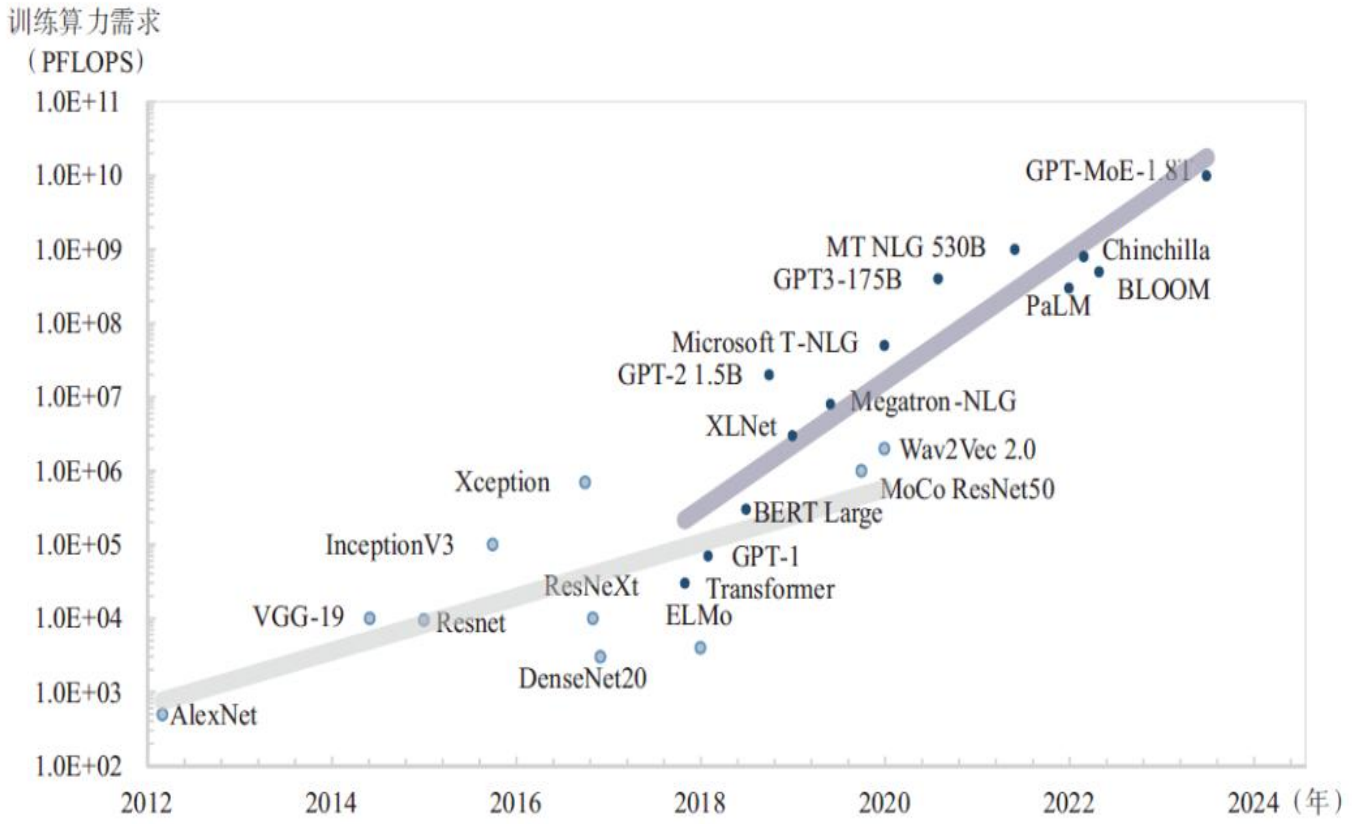


(b) Artificial Intelligence

AI发展历史与并行计算发展历史的比对

Parallel Turing Machine, A Proposal. JCST 2017.
by 高光荣（1945-2021）

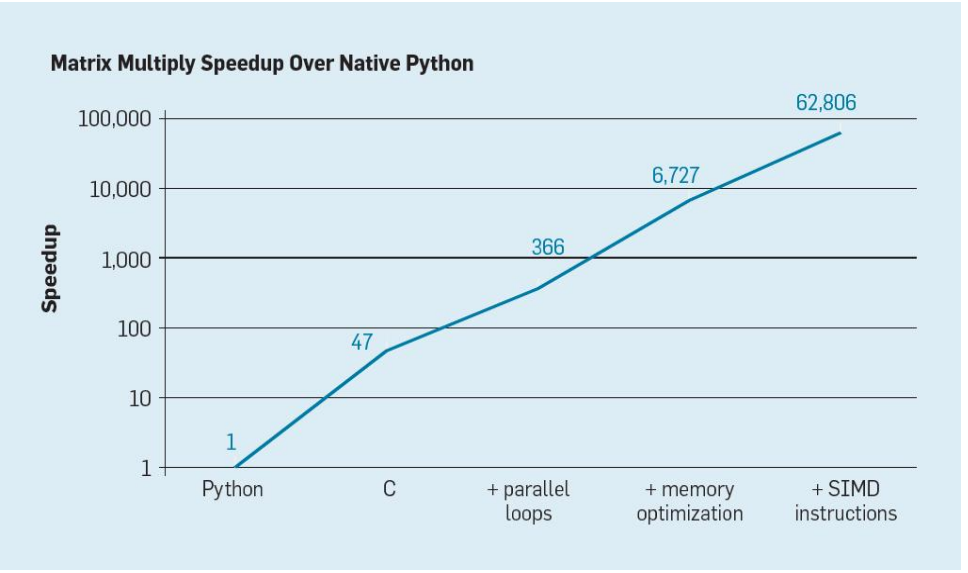
大模型对算力的需求



5 年内算力需求增长约 1,000 倍，主要由「参数规模 × 训练数据量」双轮驱动

张政, 冯少飞. 大模型算力基础设施技术趋势、关键挑战与发展路径[J]. 信息通信技术与政策, 2024, 50(6).

“All told, the final, highly optimized version runs more than 62,000x faster on a multicore Intel processor compared to the original Python version.”



From “A New Golden Age for Computer Architecture”. By John L. Hennessy, David A. Patterson, 2019

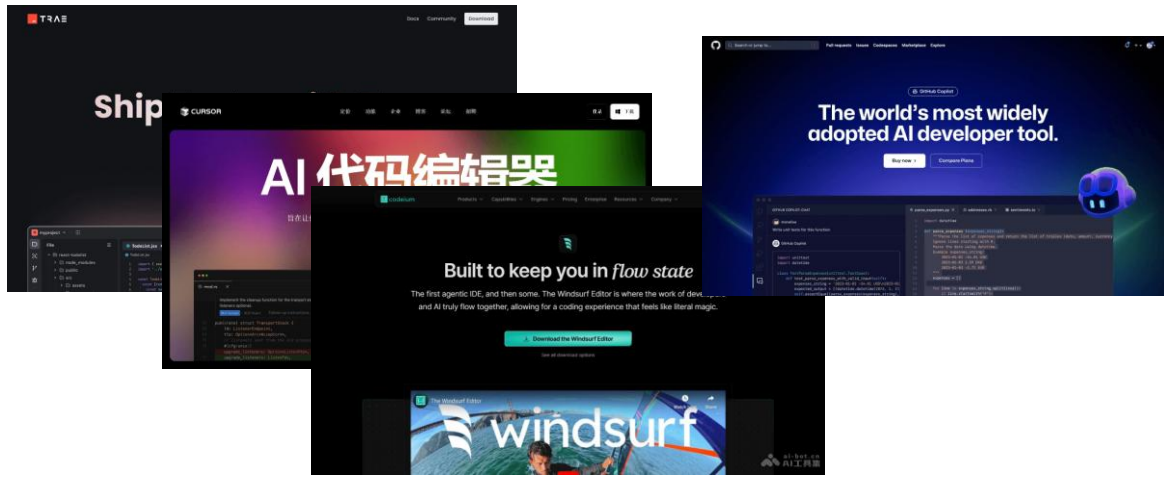


2017年图灵奖得主
计算机体系结构大师

大模型时代，计算机系统能力更重要了

AI自动化编程的普及，极大地降低了“写出能跑通的代码”的显性门槛，让更多人能快速实现想法。它同时**抬高了设计和构建真正健壮、高效、可靠、安全、可扩展的软件系统所需的“计算机系统思维”的门槛**

- Why：“能不能实现”的门槛降低了，相对的对于效率、健壮、可扩展等方面的要求就会提高
- How：要求开发者穿透AI工具提供的抽象层，深刻理解计算机系统运作的底层原理、资源的约束、组件的交互以及各种权衡取舍



计算机系统能力



清華大學
Tsinghua University



感谢各位同学积极参与

Zhang，Youhui（张悠慧）

zyh02@tsinghua.edu.cn





